

Polynomial Formal Verification of a RISC-V Processor

Lennart Weingarten[✉], Kamalika Datta[✉], and Rolf Drechsler[✉], *Fellow, IEEE*

Abstract—Verification plays a major role in ensuring the functional correctness of any design. In recent years with growing complexity of processor designs, verification has assumed utmost importance. Simulation-based techniques cannot ensure completeness in verification, and in this regard formal methods prove crucial. Although formal methods guarantee completeness it is hard to quantify the exact time and space complexities. Recently some works have demonstrated that it is possible to achieve polynomial space and time complexities for various arithmetic circuits as well as for processors. In this paper we propose a *Binary Decision Diagram* (BDD) based *Polynomial Formal Verification* (PFV) approach for verifying processors. As a case study, we discuss the PFV for a multi-cycle processor (viz., MicroRV32) with support for combinational and sequential sub-systems. New data structures and code base are utilized to verify all the functional components. Experimental results reveal that the verification can indeed be performed in polynomial time.

Index Terms—BDD, Multi-cycle, Polynomial Formal Verification (PFV), RISC-V.

I. INTRODUCTION

THE method of formal verification has drawn the attention of researchers over the last few decades as it provides a feasible alternative to ensure the correctness of a design before manufacturing. The main problem with this approach, however, is that the time and space complexities of the process are typically high and it does not scale well for larger designs. The existing approaches for tackling verification of complex designs like processors mainly rely on high-level fault models and simulation-based techniques. In recent years there has been a distinct paradigm shift and formal methods for processor verification are being explored, e.g. those based on equivalence checking, theorem proving, model checking, etc. [1], [2], [3]. Although these methods guarantee completeness in verification, the space and time complexities still remain high and unpredictable, making them undesirable for complex designs. One recent work has focused on *Statistical Model Checking* (SMC) for verifying the performance of finite-set model predictive

control (FS-MPC) algorithms. SMC provides a probabilistic answer to the verification task, indicating how well the property is satisfied by the system [4].

To tackle the above-mentioned problems, *Polynomial Formal Verification* (PFV) of digital circuits have recently drawn the attention of researchers [5], [6], [7], [8], [9], [10], [11]. In the PFV approach, the time and space complexities are guaranteed to be some polynomial function of the problem size n . However, it may be noted that it is possible to provide PFV guarantee only for specific designs, and as such it may not be possible to generalize the approach to any arbitrary design.

To this end, several formal verification approaches have been proposed in recent years [6], [9], [12], [13], [14]. These approaches can be broadly classified into two broad categories – those that try to provide PFV guarantee for existing formal verification approaches, and those that extend existing methods to guarantee PFV. Many of these methods focus specifically on arithmetic circuits like adders [5] and multipliers [14], and rely on approaches like *Binary Decision Diagram* (BDD) [15], *Symbolic Computer Algebra* (SCA), or a combination of both. However, very few works try to address PFV of more complex designs like the processor. One of the recent works [16] proposes a BDD-based PFV approach for verifying the single-cycle implementation of a selected subset of the RISC-V integer instructions. This is one of the first attempts towards achieving PFV of a complete processor and basically targets the combinational circuit blocks in the functional units for verification. Although PFV is guaranteed, the method uses complex data structures and results in higher verification times. Because of these limitations, the approach cannot be easily extended to multi-cycle processor implementations, and also cannot incorporate sequential sub-systems in the design. An attempt to support PFV on multi-cycle implementations of the RISC-V processor is proposed in [17], where a wider set of instructions including shift instructions has been considered as compared to [16]. Also, use of an improved data structure results in faster verification time. In another work [18], the authors introduce a PFV method for a multi-cycle implementation of the RISC-V processor, where all the functional units of the processor (both combinational and sequential) like Fetch, Decode, Extension, Execute and Control are covered.

In this paper, we extend the work as reported in [17], [18] to incorporate additional functionalities in the PFV process. The main contributions of this work can be summarized as follows:

- 1) The verification for the sequential sub-systems of the processor has been extended.

Received 1 January 2025; accepted 25 February 2025. Date of publication 5 March 2025; date of current version 21 March 2025. This work was supported in part by DFG within the Reinhart Koselleck Project PolyVer under Grant DR 287/36-1 and in part by the German Federal Ministry of Education and Research (BMBF) within the ECXL project under Grant 01IW22002. The review of this article was arranged by Guest Editors of the NANOARCH2023. (*Corresponding author: Lennart Weingarten.*)

The authors are with the Department of Computer Science, University of Bremen, 28359 Bremen, Germany (e-mail: len_wei@uni-bremen.de; kdatta@uni-bremen.de; drechsler@uni-bremen.de).

Digital Object Identifier 10.1109/TNANO.2025.3548265

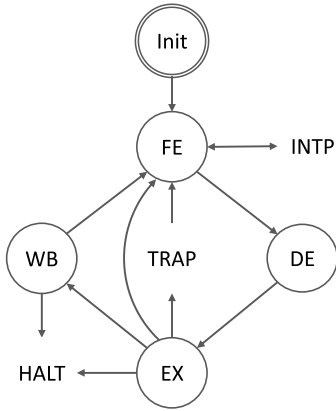


Fig. 1. MicroRV32 state machine.

- 2) The verification of the Control unit is extended with *Debug* flags, which permits diagnosis of the correct state during sequential simulation.
- 3) Additional sequential behavior has been incorporated to enable verification of the processor by reducing the circuit structure to constant control signals.
- 4) A detailed explanation of the various tools used has been provided.
- 5) An in-depth analysis of polynomial bounds for the various processor units has been included.

The rest of the paper is organized as follows. Section II provides the necessary background and a brief review of related works in this domain. Section III presents the proposed PFV methodology for processors, with details regarding the handling of the sequential sub-systems, tools used, and the generation of the reference model. In Section IV we present the experimental results followed by concluding remarks in Section V.

II. BACKGROUND

In this section we briefly discuss about the RISC-V processor, BDD-based verification and some related works on PFV techniques.

A. RISC-V Processors

RISC-V [19], [20] is a free and open-source *Instruction Set Architecture* (ISA) developed at the University of Berkeley. This is one of the most popular open-source ISA's that is widely adopted by industry and academia for various design and verification purposes. It is based on the classical load-store architecture and forms the foundation of next generation *Reduced Instruction Set Computer* (RISC) designs.

In this work we have considered the MicroRV32 processor architecture [21]. It is a multi-cycle RISC-V processor design specified at the *Register Transfer Level* (RTL) and written in a modern *Hardware Description Language* (HDL) called SpinalHDL [22]. MicroRV32 supports the RV32I base instruction set as well as the “M” and “C” extensions. In particular we focus on the verification of the RV32I base instruction set.

The *Finite State Machine* (FSM) describing the states of the multi-cycle processor is depicted in Fig. 1. The FSM allows for different pathways. Always starting in the initialization state

Init, it progresses to the instruction fetch state FE. From FE, either the interrupt state INTP or the decode state DE can be reached. After the DE state, the execute state EX will be visited, from where there are multiple options to proceed to different states, like the halting state HALT, the trap state TRAP or the write-back state WB. Furthermore, only for the S-type instructions the WB state will be visited, and for all other instructions the FE state will be visited again. Finally from the WB state the HALT or the FE states are reachable. In the present work, the HALT, TRAP and INTP states are not considered for the purpose of verification.

As mentioned, two different pathways are considered depending on the instruction, P_2 for the S-type instructions and P_1 for all other instruction types:

$$P_1 = IN \rightarrow FE \rightarrow DE \rightarrow EX \rightarrow FE$$

$$P_2 = IN \rightarrow FE \rightarrow DE \rightarrow EX \rightarrow WB \rightarrow FE$$

B. BDD-Based Verification

In this subsection, we discuss about BDD-based verification. BDD is a very popular data structure widely used for compact representation and equivalence checking of Boolean functions. A BDD is a directed acyclic graph with a designated root node and two terminal nodes labeled with constant values ‘0’ and ‘1’. Each non-leaf node is labeled with some variable (say, x), with two outgoing edges corresponding to the cases $x = 0$ and $x = 1$, respectively. We typically deal with a form of BDD referred to as the *Reduced Ordered Binary Decision Diagram* (ROBDD), where some specific ordering of the variables from the root to the terminal nodes is followed, and some reduction rules are applied to arrive at a canonical representation of the function being represented. We shall use the terms BDD and ROBDD synonymously.

It may be noted that every non-leaf node of a BDD basically specifies the function of a 2-to-1 multiplexer. Also, the BDD data structure can be used to prove the equivalence of two given functions – this is possible since the ROBDD representation of any function under a given variable ordering is unique. The *If-Then-Else* (ITE) operator [23] can be used recursively on a given function for generating the sub-functions corresponding to the BDD sub-graphs. The decomposition for one step is illustrated below:

$$ITE(f, g, h) =$$

$$ITE(x_i, ITE(f_{x_i}, g_{x_i}, h_{x_i}), ITE(f_{\overline{x_i}}, g_{\overline{x_i}}, h_{\overline{x_i}})) \quad (1)$$

The positive and negative co-factors of the variable x_i are denoted as f_{x_i} and $f_{\overline{x_i}}$ respectively. In the proposed algorithm, we use Equation (1) recursively to evaluate a BDD. The sub-graphs corresponding to the BDD nodes f , g and h are passed as arguments to the ITE function for subsequent decomposition. The worst-case computational complexity of a function using the ITE algorithm will be $O(|f| \times |g| \times |h|)$, where $|f|$, $|g|$ and $|h|$ denote the corresponding BDD sizes with respect to the number of nodes [23].

Since we deal with multi-output functions in the context of the present work, it is required to build a separate BDD for each output. For the sake of efficiency, we use symbolic simulation

to obtain the BDD's for the output functions. Basically for each output, we first generate the BDD as a function of the input variables. Then we invoke the ITE function recursively to evaluate the BDD of the function; the process is repeated till we arrive at one of the outputs. Finally, we use equivalence checking to verify whether the BDDs match against the original functional specifications.

For each functional unit *Execute*, *Decode*, *Fetch* and *Control*, golden references are provided for the purpose of checking equivalence. Due to the canonical nature of BDDs, when they are reduced and ordered, any functional equivalent but not necessary architectural equivalent Boolean function can be generated as the reference model.

C. Related Works

During the design phase, one of the major challenges is to ensure functional correctness of the design. To ensure completeness, it is imperative to use formal proof techniques for which many approaches exist; however, the time and space complexities are often unpredictable [24], [25]. To this end PFV plays an important role as it guarantees upper bounds in time and space complexities, which in turn ensure verification within certain time constraints.

Regarding PFV, the existing literature is rather sparse and very few works exist in this area. However, there have been some renewed interest in PFV in recent years. The earliest reported work in 2003 provides PFV guarantee for Wallace-tree like multipliers using *BMD-verification [26]. In later years, some works on PFV with focus on arithmetic circuits like adders [27] or simple *Arithmetic Logic Units* (ALU) [28] supporting single-cycle operations have been reported. In [14], the authors proposed a PFV approach for verifying multiplier designs using BDDs. A PFV approach to verify floating-point adders is reported in [9].

Until very recently, PFV approaches targeting complete processor designs have been largely unexplored. In [16], the authors showed polynomial upper bounds for verifying a single-cycle RISC processor design with specific focus on combinational components in the *Execute* unit. In another work [17], PFV approach for a multi-cycle processor design is discussed, where the *Decode* and *Extension* units are also included. In [18], the authors propose a PFV approach for a multi-cycle MicroRV32 processor design, and included the sequential components of the design within the domain of verification. All the functional units of the processor are included in the process of verification.

In this paper, we propose a PFV approach for the MicroRV32 processor. We exploit BDD-based verification to guarantee a polynomial upper bound. Compared to existing approaches, we extend the sequential verification to allow for a more complete verification.

III. PROPOSED PROCESSOR VERIFICATION METHODOLOGY

In this section we discuss the proposed verification methodology. We first give an overview of the approach, and then the sequential logic support. Thereafter we describe the tools developed for the verification process. Then we discuss about the reference model generation and finally a case study.

A. Overview of the Methodology

The overall verification methodology is depicted in Fig. 2. Starting with the abstract RTL specification of the MicroRV32 [21] processor expressed in SpinalHDL [22], a pre-processing step is performed for all its functional units. The functional units are first converted into Verilog using the *Scala Build Tool* (SBT). Then the pre-processing step is performed using Berkeley's ABC tool [29] and the synthesis tool Yosys [30]. The Verilog representation is synthesized into an *And Inverter Graph* (AIG) representation.

Specifically, the following steps are performed, as explained in Fig. 2.

- Each subsystem of the processor, represented in SpinalHDL, is converted to Verilog and then to AIG representation during the preprocessing step.
- For verifying the various subsystems, different methods are used. For instance, for verifying the *Fetch* and *Control* units, we use partial and symbolic simulation in conjunction with sequential simulation. Similarly, for the *Decode*, *Extension* and *Execute* units, we use partial simulation. In this regard we perform verification of both combinational and sequential logic. In *Partial Simulation* (PSIM), we use ternary values ('x', '0', '1') to simulate a circuit, following which the functionality is extracted and the graph size reduced. And in *Sequential Simulation* (SSIM), we extend the ternary simulation over multiple cycles to emulate the sequential behavior. We use *Symbolic Simulation* (SYMSIM) to generate equivalent BDD representations, where symbolic variables are used instead of Boolean values.
- As mentioned, we use SYMSIM to generate equivalent BDD representations, and also create the reference BDDs using a reference BDD generator. For the final verification, the extracted BDDs are compared with the reference BDDs to determine whether the representations are equivalent.

The detailed steps followed for the various functional subsystems are summarized below.

- For the *Fetch* unit, we identify three different cases of sub-circuits that are extracted using PSIM for subsequent verification using SYMSIM.
- For each instruction type, we use PSIM to extract sub-circuits corresponding to the *Decode*, *Extension* and *Execute* units, and then use SYMSIM for final verification.
- For the *Control* unit, we model the overall behavior of the processor as a FSM comprising of various steps in instruction execution like initialization (Init), fetch (FE), decode (DE), execute (EX) and result writeback (WB). Two alternate pathways are possible during instruction execution. For instance, for the S-type instructions, we follow the sequence:

$$P_1 :: \text{Init} \rightarrow \text{FE} \rightarrow \text{DE} \rightarrow \text{EX} \rightarrow \text{WB} \rightarrow \text{FE}$$

whereas all other instructions we follow the sequence:

$$P_2 :: \text{Init} \rightarrow \text{FE} \rightarrow \text{DE} \rightarrow \text{EX} \rightarrow \text{FE}$$

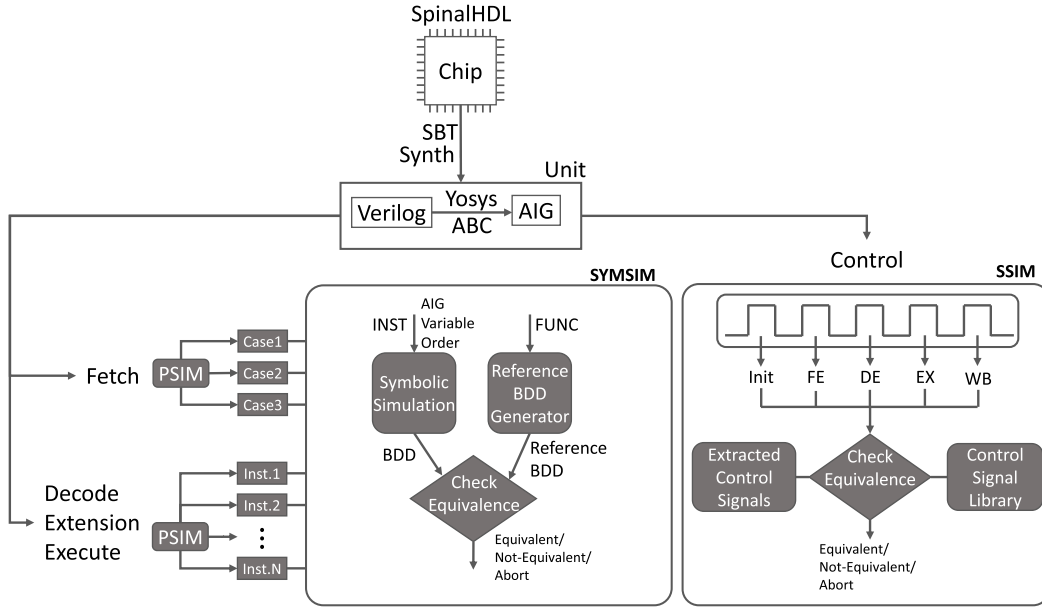


Fig. 2. Methodology overview.

- d) For all the extracted sub-circuits we use SYMSIM to generate the BDD except for the control unit. Reference models are generated for the extracted sub-circuits as well. Lastly the generated BDD is compared with reference BDD. For the control unit, an equivalence-based checking method is employed to verify the extracted control signals for each state.

B. Sequential Logic Support

It may be recalled that in [17], processor verification is targeted assuming multi-cycle implementation and considering only combinational circuit modules. In a subsequent work [18], sequential circuit modules like latches are incorporated with consideration of multi-cycle implementation. In this paper, we further extend the capabilities of the tool to provide full sequential logic support.

To provide support for sequential logic elements, PSIM has been extended to provide sequential support (referred to as SSIM). The parser that analyzes the AIG representations has also been extended to support latches. We consider two types of latches, with and without reset. To support sequential elements, a new node type for latch has been added in the graph structure for the simulation code base. Also, we have separated out the phases of graph reduction and multi-cycle instruction simulation, which allows simulation to be carried out before graph reduction.

The following are the specific extensions that have been incorporated to include sequential support:

- a) The design of the parser that preprocesses the specification of the ISA has been modified to include support for sequential elements like latches.
- b) To incorporate support for latches, we have added two new cases that serve the purpose of including the description of sequential elements in the AIG.

- c) The format of the AIG header has been extended to include support for latches, which was missing in the previous work.
- d) A new node type has been added to the graph structure for specifying latches. This updated data structure can elegantly model the sequential elements in the design.
- e) The phases of graph reduction and multi-cycle simulation have been separated out for the purpose of simulating the complete instruction set. This allows multi-cycle simulation to be carried out before the phase of graph reduction.

Fig. 3 shows the example of an *Universal (left/right) Shift Register* (USR) containing memory elements. It may be noted that the process of handling sequential behavior in circuits is handled by unrolling sequential circuits up to a limited sequential depth. The USR has a 4-bit data input `in`, a select signal `sel` to select between left or right shift, a most significant bit `msb` and least significant bit `lsb` input to fill in the space after a right/left shift, and a `clear` signal to reset the register. Four latches `lat`, and the corresponding primary outputs called `out` are present in the circuit. The clock signal `clk` is separated out from the circuit, as it is no longer needed due to the cycle-accurate design of the AIGs.

C. Tools

All the tools developed for this work are compatible with the *And Inverter Graph* (AIG) representation. For instance, the tools PSIM and SSIM are used to extract functionality from a circuit represented as an AIG. To verify the extracted functionality the tool SYMSIM is used. In this subsection we shall discuss about the tools in some detail.

To extract the functionality from a circuit, a symbolic simulation approach is chosen in this work. For this purpose the PSIM tool is developed. For example, the extraction for the ADD instruction is performed by applying 2-bit vectors to the circuit,

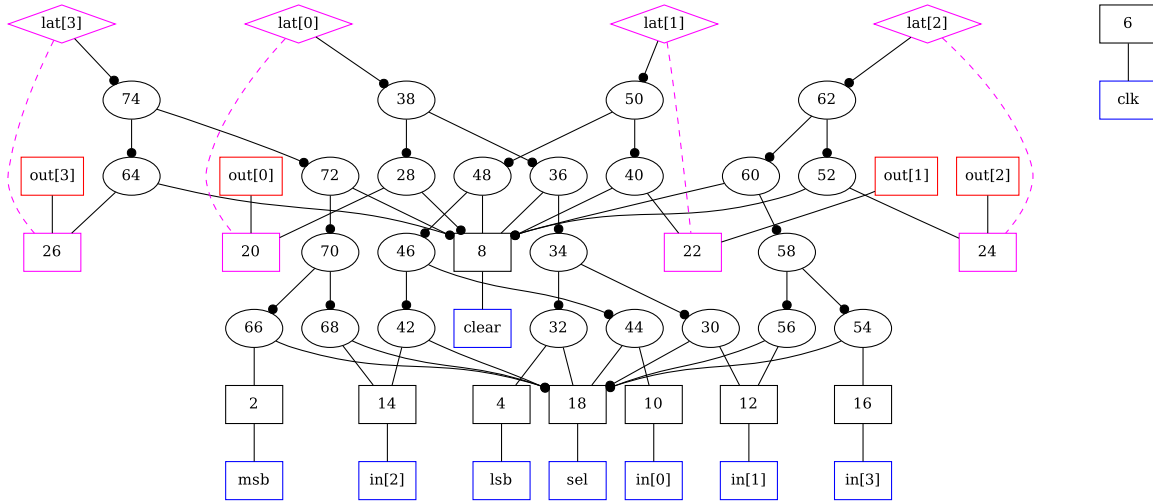


Fig. 3. 4-bit universal left/right shift example with memory elements.

one describing a set of stimuli for the primary inputs and the other describing the states of the latches at the start of simulation. Both vectors contain ternary-valued elements, viz. '0', '1' and 'x'. The stimuli vector also contains ternary values depending on the functionality that should be extracted. The data values are set to an unknown value and the control signal to select the functionalities are set to either 0 or 1. After the stimuli and latch contents are applied to the circuit, it is evaluated level by level until the primary outputs of the circuit are reached. Then to extract the functionality, the circuit is reduced in size by removing all nodes except the primary outputs, which contain a constant value 0 or 1. Also fanins which lead to constant values are removed. This results in a minimized circuit describing only the desired functionality.

For combinational circuits the vector defining the latch states is not needed and is therefore empty. It is used only when the circuit contains sequential elements like latches. To extract functionality of circuits with latches, multiple clock ticks might have to be simulated to arrive in a desired state. For this purpose the SSIM tool has been developed. It extends PSIM with support for sequential elements. In the AIG representation of the processor, the clock signal is no longer necessary due to AIG's cycle-accurate model. For example, to reach the `decode` state in Fig. 1, initially the stimuli and the vector for latches are prepared for the `init` state and the circuit are simulated once, the clock is advanced once, and the `Init` state is reached. Then a stimuli set for the `Fetch` state is applied and no vector for the latches must be provided. Once it is initialized, the latch state consistently updates over time. For further simulations, no latch vector is needed. By simulating it second time the clock is advanced again and the `Fetch` state is reached. Finally a set of stimuli to prepare the `Decode` state is applied and simulated, and the desired `Decode` state is reached. After reaching the desired state, the circuit graph is reduced and functionality for the desired state is extracted.

After the functionality is extracted, the SYMSIM tool is used for verification. For the extracted functionality, SYMSIM traverses the circuit from the primary inputs creating a BDD

using the CUDD [31] library. In each iteration the ITE-operator is used, resulting in a canonical BDD representation of the extracted functionality. This BDD is then compared with a golden reference model using equivalence checking. However, during equivalence checking it must be ensured that the variable ordering for the extracted functionality and the golden reference model are the same.

D. Reference Model Generation

The task of providing a golden reference model for verification is performed by the *Reference Model Generator* (RMG) tool. RMG is one of the vital components in the verification process. For the MicroRV32 processor used in this work, a reference BDD is created for each of the instructions used for verification. A golden reference BDD is generated directly from the mathematical description of the functionality considered in this work. In general, a reference circuit is not required in this approach. For example, a BDD can be constructed for the logical exclusive-or with n bits, which can then be compared with the BDD extracted from the *Execute* unit. For the verification to be successful, the BDD nodes of the primary inputs of the reference model must be shared with the BDD of the instruction under verification. A library consisting of all the reference BDDs is constructed. As mentioned before, functionally equivalent designs for individual instructions are considered for this purpose. Also, the RMG is responsible for generating all the reference BDDs for all the outputs. As the complexity of BDDs is greatly influenced by variable ordering, we make use of this to generate efficient BDDs.

The `Fetch` unit includes three cases for which the reference models are generated. The first case is reset state, the second state is reading from the buffer, and the third state is reading and writing into the buffer. Depending on the case, a BDD is generated for each of the outputs. The first case consists of the reset state for `Fetch`, where the resulting data only consist of the reset value 0. The second case is the direct connection of the

input data to the output of the Fetch unit. And in the third case the output values of the Fetch unit are read from a buffer.

For the Decode, Extension and Execute units, we generate the reference model for each of the instructions. Broadly various types of arithmetic, logical and shift operations are considered. For the logical instructions like (OR, XOR, and AND and their immediate analogue, e.g. ORI, XORI, and ANDI), we employ a bitwise logical operation for reference model creation.

For the shift instructions (SLL, SRL etc.) we implement a generic shift operation. The reference BDD is generated for all types of shift operations involved.

Among the various arithmetic instructions, the addition operation is most common. An adder is required for ADD and ADDI. For jump instructions like JAL and JALR, to generate the target address an offset is added with the program counter. Similarly for LOAD or STORE instructions, the effective address is calculated by adding the register content with an offset value as specified in the instruction. The reference BDDs are generated for all such instructions using the adder function. Also for addition, the BDD for the carry flag is generated as the one corresponding to the final carry of the result.

Subtraction of one operand from another is performed using SUB instruction. There exists many instructions that involves subtraction. For branch instructions (like BEQ, BNE, BLT and BGT), some comparison is required that requires subtraction followed by checking the appropriate flags. Also for various SET instructions like (like SLT, SLTU, etc.) a similar structure is required. For all such instructions an implementation of subtraction is used for generating the reference BDD.

For the Control unit we do not require any reference BDD generation. Rather for each instruction type, the current state of the FSM is matched with the corresponding golden response. A library is generated for the control signals for each instruction type corresponding to each state. SSIM is used to extract the control signals of each state, which in turn is matched with those stored in the library.

E. Case Study: XOR-Instruction Verification Using BDDs

We now discuss the steps required to verify the XOR-instruction, which is part of the Execute unit, starting from stimuli generation to functionality extraction as the BDD for the function, the reference BDD, and finally the BDD comparison.

Step 1 (stimuli generation): Before the XOR-instruction can be verified, it has to be first isolated from all the other parts of the Execute unit. For the isolation/extraction of functionality partial simulation is performed. For this a set of stimuli is defined, describing all the control parameters needed to select the XOR-instruction. These parameters are defined in the RISC-V standard for integer base instruction set RV32I. The stimuli in Fig. 4 describes the values for both the operands A and B and the control parameters for opcode, func3 and func7. All the control parameters are set to constants “0” or “1” and the operands for A and B to unknown value “x”.

These stimuli parameters for all RV32I instructions are stored in a JSON configuration file and then read by the PSIM tool, which selects the appropriate parameters for each instruction.

```

xxxxxxxxxxxxxxxxxxxx xxxxxxxxxxxxxxxxxxxx # opA
xxxxxxxxxxxxxxxxxxxx xxxxxxxxxxxxxxxxxxxx # opB
100                                         # func3
00000000                                   # func7
0110011                                   # opcode

```

Fig. 4. Stimuli for XOR-instruction.

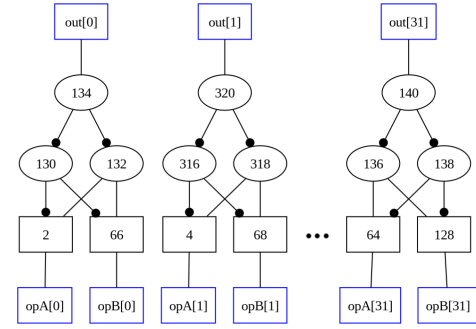


Fig. 5. Excerpt of reduced Xor and inverter-graph of four primary outputs.

Step 2 (graph reduction): To isolate the XOR-Instruction from the rest of the Execute unit, the stimuli from Fig. 4 is applied to the primary inputs of the AIG of the Execute unit. Then the PSIM tool is used to perform a level-by-level evaluation of each node in the graph until the primary outputs are reached. Next the graph is reduced by removing all the nodes containing constant “0” or “1”. Only the nodes holding the unknown value of “x” remain. This procedure removes all the control signals and structures and only the core functionality describing the XOR operation remains in the graph. The reduced graph only contains the functionality of the isolated XOR-instruction. Fig. 5 shows an excerpt of the entire graph, for the primary output 0, 1 and 31.

Each primary output (out) has a Xor-graph structure consisting only of AND and NOT gates starting from two primary inputs. From the figure it can be seen the graphs for each of the primary outputs are independent.

Step 3 (BDD generation): For each of the primary outputs of the reduced graph for the XOR-instruction a BDD is generated. This step is performed using our SYMSIM tool utilizing the CUDD library. For all the BDDs generated, an interleaved variable order is used. For example, the first primary output out[0] is generated by construction the BDDs using the ITE-Operator from Equation (1). Equation (2) shows each step starting with assigning the two primary inputs opA[0] and opB[0]. The variable names in this example have been chosen to match the AIG node names from Fig. 5. After this the intermediate BDDs for the nodes 130 and 132 are created. Then the node 134 combines both 130 and 132 BDDs into one. Finally the out[0] is matched to the BDD 134.

$$opA[0] = 2$$

$$opB[0] = 66$$

$$130 = ITE(-2, -6, 0) = -2 \wedge -6$$

TABLE I
TRUTH-TABLE OF THE INTERMEDIATE AND FINAL RESULTS OF THE
ITE-OPERATION APPLIED ON THE XOR AIG

opA[0] 2	opB[0] 6	intermediate 130 132		out[0] 134
0	0	1	0	0
0	1	0	0	1
1	0	0	0	1
1	1	0	1	0

$$132 = ITE(2, 6, 0) = 2 \wedge 6$$

$$134 = ITE(\neg 130, \neg 132, 0)$$

$$= ITE(\neg ITE(\neg 2, \neg 6, 0), \neg ITE(2, 6, 0), 0)$$

$$= ITE(\neg(\neg 2 \wedge \neg 6), \neg(2 \wedge 6), 0)$$

$$= \neg(\neg 2 \wedge \neg 6) \wedge \neg(2 \wedge 6)$$

$$out[0] = 134 \quad (2)$$

The truth-table for the intermediate and final AIG nodes are presented in Table I. From the last column in the table it can be seen that in fact this BDD indeed models the XOR function.

Step 4 (reference BDD generation): Before the verification step a reference BDD is generated using the *Reference Model Generator* (RMG). Due to the canonical properties of BDDs we can generate a XOR function that is functionally equivalent, but must not be structurally equivalent. The XOR-instruction models a bit-wise XOR over all the input bits of operand A and B. For example, we can define the reference BDD for any output at index i as:

$$a_i \oplus b_i = ITE(a_i, \neg b_i, b_i) = (a_i \wedge \neg b_i) \vee (\neg a_i \wedge b_i)$$

Step 5 (BDD comparison): To be able to compare the BDD of the function being verified and the reference BDD easily, the BDD nodes of the primary inputs are shared between both the BDDs. The BDDs of each of the primary outputs are compared against the reference BDDs. This is done by comparing the pointers to the output nodes of the reduced graph and reference BDD; if all of the pointers match the entire XOR-instruction is verified.

IV. EXPERIMENTAL EVALUATION

All experiments have been performed on a system with the following configuration: Intel(R) Core(TM) i7-8565 U CPU@1.80 GHz clock with 16 GB of main memory. All the tools, SSIM, PSIM and SYMSIM, are implemented in C++. In the following, the experimental results are presented in Section IV-A followed by a polynomial analysis in Section IV-B and finally limits of the approach are discussed in Section IV-D.

TABLE II
RESULTS OF FETCH UNIT

Nr.	Cases	PSIM [ms]	PFV [ms]	Nodes
i	Reset Buffer	0.46	0.04	65
ii	Read Buffer	0.43	0.04	33
iii	Read & Write Buffer	0.41	0.05	65
Total		1.30	0.13	163

A. Verification Results

Results are presented for all functional units, Fetch and Control units as well as the Execute, Decode-and-Extension unit (DEU), of the MircoRV32 processor. All the experiments are run multiple times and the results averaged out to give a more conclusive depiction of the performance. The average values are reported as the final result.

The verification of the Fetch unit containing a buffer requires the consideration of three different functional cases. Case-i is for resetting the buffer, case-ii for reading from the buffer, and case-iii for both reading and writing from the buffer. The sub-circuit for each of these cases is extracted and evaluated individually. For case-i, all primary outputs have to be evaluated to a constant 0. Case-ii will result in the value writing into the buffer in a previous time frame. And reading and writing to/from the buffer in case-iii will replace the current values stored in the buffer and also place the current values from the buffer onto the primary outputs. This behavior has been modeled and verified. For this the PSIM tool is used to first extract the three functional cases and subsequently the SYMSIM tool is executed to verify the extracted functionality.

SYMSIM allows for the verification of sequential circuits, by placing variables in form of a BDD on to the primary inputs of the circuit and then by performing simulation of the variables that are iteratively passed through the circuit. Evaluation of the circuit is carried out by verifying whether the variables are in a specific position as primary output or latch in the circuit, and are in the expected order after a given number of simulated clock cycles.

Results for the Fetch unit are presented in Table II. From left to right, the first column Nr represents the case number followed by the case name in Cases column. Then in column PSIM and PFV the functional extraction and verification times in milliseconds are presented. In the last column Nodes, the number of BDD nodes needed for verification is shown. For case-i the PSIM time is longer compared to the other two cases. The reset case requires a longer reduction phase of the circuit to yield the extracted functionality. Compared to case-ii and case-iii the entire circuit graph is reduced to a constant vector of 0's. Also the PFV time for the reset case involves the generation of BDDs as a golden reference model. And for the other two cases we only have to consider the BDD variables as pass-through values and perform equivalence checking at the end. The number of BDDs in Nodes are explained as follows: for case-i and case-iii, 32-bits are used for both the primary inputs and outputs in addition to a terminal BDD node for a total of 65 BDD nodes. Case-ii is the read case where 32-bits for the primary inputs

TABLE III
RESULTS OF CONTROL UNIT STAGES

Stage	Type	Option	SSIM [ms]	PFV [ms]	DVerf
Init	All	-	0.91	0.067	0000
FE	All	normal	0.96	0.079	0001
	All	interrupt	0.94	0.083	0001
DE	All	-	0.95	0.071	0010
EX	R	normal	0.97	0.089	0011
	R	sub/sra	0.99	0.089	0011
	I	regImm	0.96	0.084	0011
	I	immImm	0.97	0.089	0011
	B	-	0.99	0.091	0011
	J	jal	0.99	0.088	0011
	J	jalr	1.00	0.087	0011
	U	lui	0.98	0.087	0011
	U	auipc	0.96	0.087	0011
	S	lb	0.95	0.085	0011
	S	lh	0.99	0.095	0011
	S	lw	0.98	0.086	0011
	S	lbu	0.96	0.087	0011
	S	lhu	0.98	0.088	0011
	S	sb	0.98	0.097	0011
	S	sh	0.97	0.091	0011
	S	sw	0.98	0.091	0011
WB	S	lb	0.96	0.089	0100
	S	lh	1.01	0.089	0100
	S	lw	0.99	0.092	0100
	S	lbu	0.99	0.097	0100
	S	lhu	0.98	0.090	0100
	S	sb	0.97	0.087	0100
	S	sh	1.00	0.085	0100
	S	sw	1.00	0.087	0100
Total			28.26	2.527	✓

result in 32 BDD nodes and a terminal BDD node resulting in a total of 33 BDD nodes. The number of peak BDD nodes is not listed in the table, it requires 68 peak nodes for each of the three cases.

The verification of the `Control` unit requires the consideration and verification of all its states. The `Control` unit models a FSM, in which two different pathways as mentioned previously are considered (Section III-A). To verify the `Control` unit, the circuit has to be split into its corresponding FSM states. Due to the sequential nature of the FSM the functional extraction of control signals for the `Control` unit is carried out using the SSIM tool. Extracted control signals are then compared to a library of golden references, which are extracted from the specification.

Table III presents the results of the `Control` unit under the consideration of FSM states and for all the different RISC-V instruction types. From left to right the columns of the table present the FSM stages in `Stage`, the instruction type in `Type` and a variation of the instruction type in `Option`. Then the subsequent columns present the SSIM time for functional extraction and the verification time PFV, both in milliseconds. The last column `DVerf` presents the verification of the stage, by verifying debug outputs generated according to the state the FSM is currently in. The value present in that column represents the values generated for each of the four debug outputs. The results show that each FSM state could accurately be reached through simulation. This is in addition to the verification process

TABLE IV
RESULTS OF CONTROL UNIT

InstTy.	Path	SSIM [ms]	PFV [ms]	#CtrlS	DVerf
R	P_1	5.72	0.478	192	✓
I	P_1	5.69	0.473	192	✓
S	P_2	19.45	1.736	640	✓
B	P_1	4.75	0.391	160	✓
U	P_1	5.70	0.474	192	✓
J	P_1	5.75	0.475	192	✓
Total		47.06	4.027	1568	✓

as shown in PFV. In the following the results (`DVerf`) for all the FSM stages are discussed.

Init For the `Init` state different instruction types do not have to be considered; the stage is same for all of them. Therefore the `Type` in the table is listed as “all”. The expected primary debug outputs of “0000” are reproduced from the sequential simulation.

FE The `Fetch` stage consists of two different cases, and like `Init` the generated control signals are the same for all instruction types. Only the difference between the “normal” and “interrupt” cases have to be considered. For this the simulation creates the control signals for an interrupt case and the normal case of the `Fetch` stage. Expected debug state is “0001” and again it has been generated by SSIM.

DE In the `Decode` stage, like `Init` there are no different cases, and all instruction types produce the same control signals. `DVerf` matches the expected “0010” debug outputs of the `Control` unit.

EX The `Execute` stage needs to be considered not only for all the RISC-V instruction types, but also when each time there is some variants/options that generate slightly different control signals. For example for the I-type the variation of both operands are immediate values option “immImm”, or if one operand comes from a register and the other one is provided as an immediate value “regImm”. All instruction types except the R-type take roughly the same time for extraction and verification. The S-type load and store instructions have to be considered separately and cannot be verified as a whole. This is because some of the `Control` units outputs highly depend on the bit-width (byte, halfword or word) considered by the instruction, viz. the *strobeSelect* control signal of the internal memory control unit.

WB In the write-back stage only the S-type instructions have to be verified; all other instruction types do not encounter this FSM stage. Similar to the `Execute` all the S-type instruction have to be considered.

The extraction and verification for the `Control` unit takes a total of 28.26 milliseconds, and all FSM stages can be extracted accurately using sequential simulation using our SSIM tool.

In Table IV the results for the `Control` unit as per the two different pathways P_1 and P_2 are presented. From left to right the first two columns respectively list the instruction type `InstTy` and the FSM `Path` used for each instruction. The time for

the sequential functional extraction measured in milliseconds is presented in column *SSIM* and the time for the polynomial formal verification in column *PFV*. Column *#Ctrls* lists the number of control signals and last column *DVerf* shows if the targeted FSM state could be verified by the debug control signals. For each instruction type all the visited stages in the FSM and also options/variants from Table III have to be considered to get the total time for the extraction and verification of each instruction type. All types except the S-type uses the path P_1 ; the writeback state is only present in the path P_2 that is only used by the S-type.

Compared to [18] where only 15 out of the 32 outputs/control signals of the *Control* unit could be considered, in this work all 32 control signals are verified. In previous works the extraction of the functionality was not able to fully minimize and extract all control signals to a constant value, which is now possible with the current improvements in the *SSIM* tool. Now the behavior and simulation latches are optimized and results in a minimized result of the simulation. All primary outputs of the *Control* unit collapse during the simulation to a constant value. From the results it can be inferred that the *SSIM* tool accurately creates the desired FSM state after a given number of clock cycles. Results from Table IV show the entire flow of each instruction type through the FSM of the *Control* unit that can be verified in about 51 milliseconds, taking about 47 milliseconds for the extraction of functionality and around 4 milliseconds for the verification process itself. Due to the necessity to verify all the instructions for the S-type separately, the S-Type and thereby the P_2 path takes the majority of the extraction time and also takes the longest for verification.

The experimental results for the *Execute*, *Decode*-and-*Extension* unit (DEU in short) are presented in Table V. Because the *Extension* unit is part of the *Decode* unit, it is not verified separately.

From left to right the first two columns show the instruction category *G.* and the actual instruction *Inst.* The instruction group represents the nature of the operation being carried out by the instruction. Then in columns 3 to 6 the results for the *Execute* unit and in columns 7 and 8 the results for the DEU are presented. Columns 3 and 7 show the partial simulation times in milliseconds for the *Execute* and DEU units. The verification times are presented in columns 4 and 8 respectively, also in milliseconds. For the *Execute* unit, the number of BDD nodes and BDD peak nodes observed during BDD creation are presented in column 5 *Nodes* and column 6 *Peak*. For the DEU the number of BDD nodes and BDD peak notes in not listed, since they are the same for all the instructions – 65 for the BDD size and 68 for the BDD peak size. Also not explicitly shown in the table are the total number of BDD nodes and peak BDD nodes, which are 65 and 68 respectively.

From top to bottom, first the results for all the logic based instructions like AND, OR, XOR and their immediate counterparts are presented, followed by the results of all the shift operations. Then all the results for the addition and subtraction based instructions are presented.

Form the results it is clear that the Addition based instructions take the longest for the extraction *PSIM* time followed by the

TABLE V
RESULTS OF EXECUTE, DECODE- AND EXTENSION UNIT (DEU)

G. Inst.		Execute				DEU	
		PSIM [ms]	PFV [ms]	Nodes	Peak	PSIM [ms]	PFV [ms]
Logic	AND	4.28	0.06	97	100	0.75	0.10
	ANDI	4.18	0.06	97	100	0.71	0.11
	OR	4.21	0.06	97	100	0.72	0.10
	ORI	3.98	0.06	97	100	0.73	0.11
	XOR	5.11	0.10	161	164	0.71	0.10
	XORI	5.00	0.09	161	164	0.71	0.11
Shifts	SLL	4.64	0.59	1632	1635	0.66	0.09
	SLLI	3.81	0.56	1632	1635	0.55	0.08
	SRL	4.44	0.61	1431	1434	0.71	0.10
	SRLI	4.30	0.61	1431	1434	0.66	0.09
	SRA	4.52	0.71	1396	1399	0.71	0.10
	SRAI	4.34	0.69	1396	1399	0.68	0.10
Addition	ADD	4.05	0.31	1327	1330	0.51	0.07
	ADDI	4.09	0.30	1327	1330	0.52	0.09
	JAL	4.07	0.32	1327	1330	0.52	0.08
	JALR	3.91	0.31	1327	1330	0.51	0.08
	AUIPC	4.92	0.36	1327	1330	0.65	0.09
	LUI	4.18	0.32	1327	1330	0.53	0.08
	LB	3.86	0.30	1327	1330	0.54	0.08
	SB	4.10	0.32	1327	1330	0.55	0.09
	LW	4.59	0.23	804	807	0.70	0.11
	SW	4.88	0.22	804	807	0.71	0.11
	LH	4.49	0.24	804	807	0.66	0.10
	SH	4.67	0.23	804	807	0.67	0.10
LBU	4.87	0.23	804	807	0.72	0.11	
LHU	4.92	0.24	804	807	0.73	0.11	
Subtraction	SUB	4.61	0.38	1415	1418	0.61	0.09
	BEQ	8.01	0.15	401	404	0.52	0.09
	BNE	8.52	0.15	401	404	0.65	0.09
	BGE	8.07	0.20	951	954	0.72	0.11
	BLT	8.00	0.21	951	954	0.68	0.11
	BLTU	7.99	0.21	947	950	0.72	0.11
	BGEU	8.16	0.19	947	950	0.72	0.11
	SLT	8.52	0.59	992	995	0.68	0.10
	SLTI	8.72	0.60	992	995	0.66	0.09
	SLTU	8.07	0.56	809	812	0.69	0.10
	SLTIU	8.33	0.58	809	812	0.69	0.11
	Total	201.41	11.95	34683	34794	24.16	3.60

Subtraction based ones. The Shift and Logic operations take the least amount of time for the functional extraction *PSIM* and verification *PFV*.

In total the extraction time of the *Execute* unit for all the instructions can be performed in 201.41 milliseconds and the verification of the entire *Execute* unit could be carried out in 11.95 milliseconds. DEU unit needs 24.16 in total for extraction and 3.60 for verification. In summary, the entire functional extraction and verification can be performed in very less time.

B. Polynomial Analysis

In this subsection we provide the polynomial time complexity for verification of all the processor stages. For the *Fetch* stage three cases need to be verified. The time complexity of the reset operation is $\mathcal{O}(n)$ for an n -bit processor. This involves resetting the register related with the *Fetch* stage, thereafter checking the logic states of the required bits from the register in an iterative way. For the read operation the time complexity is $\mathcal{O}(n)$. As the read operation require checking the logic value stored in the

buffer by a previous write operation, it requires simulation of a given input value to write back in the buffer in constant time (i.e., $\mathcal{O}(1)$), and thereafter reading the n -bit buffer state to compare with the input bit-vector in $\mathcal{O}(n)$ time. For the third case, i.e. read and write operations together, involves repeated simulation of assigning an input value to the buffer and then reading it from the buffer state in which the primary inputs are compared to the references resulting in a complexity of $\mathcal{O}(2n)$, due to the need of comparing the input values twice. The resulting polynomial verification complexity for the `Fetch` unit is:

$$\mathcal{O}(n) (\approx \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(2n)) \quad (3)$$

The complexity class for the `Execute`, `Decode-and-Extension` unit is as follows:

$$\mathcal{O}(n^2) \left(\approx n \cdot |s| \cdot \sum_{i=0}^{n-1} |A_i| \cdot |B_i| = n^2 \cdot |s| \cdot 3^2 \right) \quad (4)$$

Depending on the type of operands, viz. register-register type (e.g., `ADD`, `SUB`, etc.) or register-immediate type (e.g., `ADDI`, `SLLI`, etc.) as well as instruction types: `R` – register, `I` – immediate, `S` – load and store, `B` – branch, `U` – upper immediate, and `J` – jump, different scenarios must be considered for verifying `Decode` and `Extension` units. To differentiate between various register-immediate type from register type instructions, SYMSIM uses a bit-wise MUX as part of the `Extension` unit. It is known that translation of a MUX for a single variable ITE operation results in BDD of size 3. But for 3 variables, $ITE(s, A_i, B_i)$ results in a BDD of size 7. Taking into account the ITE analogy of SYMSIM in [16], the total verification complexity is polynomially bounded by Equation (4).

The verification of `Execute` unit is determined by the type of operations: *Addition*, *Subtraction*, *Logic* and *Shift*. The overall run-time complexity of SYMSIM for *Shift* and *Logic* group of operations is bounded by $\mathcal{O}(n)$, and for *Addition*, it is bounded by $\mathcal{O}(n^2)$. For the *Subtraction* group, we first need to redefine it in terms of 2's complement addition, i.e. $A - B = A + B' + 1$, where B' is 1's complement of B . As it is already shown that *Addition* group of instructions are bounded by $\mathcal{O}(n^2)$, we can convincingly say that *Subtraction* group attains similar bounds as well. Hence for the `Execute` unit the polynomial run-time complexity is bounded by $\mathcal{O}(n^2)$.

For the `Control` unit verification we need to first consider SSIM and its FSM states, i.e. $FE \rightarrow DE \rightarrow EX \rightarrow WB$. Thereafter the evaluation of control signal status analogous with a particular state against the golden reference is performed. Taking into account the simulation of each FSM state and verification of control signal status as atomic, i.e. ($\mathcal{O}(1)$), the verification of control signals for m states of a processor is $\mathcal{O}(m + m) \sim \mathcal{O}(m)$. Hence for k number of instructions the verification complexity has a polynomial bounds of $\mathcal{O}(km)$.

Table VI shows the overall result and corresponding polynomial class. The extraction time and verification time (in milliseconds) are shown in columns 2 and 3. The last column shows the polynomial class for each unit of the processor. The complete verification of the 32-bit integer base instruction set of the MicroRV32 processor is performed within 294 milliseconds,

TABLE VI
OVERVIEW RESULTS

Units	Extraction Time [ms]	Verification Time [ms]	Polynomial Class
Execute	201.41	11.95	$\mathcal{O}(n^2)$
DEU	24.16	3.60	$\mathcal{O}(n^2)$
Fetch	1.30	0.13	$\mathcal{O}(n)$
Control	47.06	4.03	$\mathcal{O}(km)$
Total	273.93	19.71	$\mathcal{O}(n^2)$

where majority of the time is spent on extraction and about 20 milliseconds for verification. The polynomial class or complexity class for `Fetch` is $\mathcal{O}(n)$, for `Execute` and `DEU` are $\mathcal{O}(n^2)$, and for the `Control` unit it is $\mathcal{O}(km)$, where n is the number of inputs, k is the number of instructions and m is the number of states that are required to be stimulated.

C. Comparison With Previous Work

In this section results are compared with one previous work [16]. The processor considered in [16] is a *Single-Cycle* (SC) RISC-V processor whereas in this work a *Multi-Cycle* (MC) RISC-V processor is considered. For the SC the entire processor was evaluated as a single execute unit, whereas the MC could be split into various functional units like `Execute`, `Fetch`, `Decode`, etc. and can be verified without any modifications to the processor. Therefore, only *Execute* unit in this work can be compared with [16]; Table I for the previous work and Table VI for the current work. Also, the SC was modified to remove all sequential elements like registers to make it fully combinational, whereas the MC does not require any such modification.

Like previous work the RISC-V instructions supported by the *Execute* unit are grouped into logic, shift, addition, and subtraction instructions. Comparing the node size and peak node size of previous and current results, we find that MC has consistently smaller node and peak sizes for all instructions except the shift instruction. This difference comes from the approach chosen to verify the shift operations in [16]. [16] considers all possible shifts, which needs to be verified for all possible input/output mappings, resulting in a number of BDD nodes of the input size. The current work verifies each shift instruction once, by providing a general shift reference model. This drastically reduces the memory and time usage to verify all shift instructions. However, it increases the number of nodes required to model the shift.

Comparing node sizes for the logic-based instructions like `AND`, `OR`, and `XOR`, the SC requires more gates than the MC processor. The larger node sizes are due to the extra output flags for zero, sign and carry as well as an extra program counter source output for the SC, which are not present in the MC implementation of the *Execute* unit.

Because of the differences in SC and MC, only the PSIM and PFV times can be compared. The PSIM time of the SC was about 969 s, whereas for MC it is only 204 ms. This improvement in performance is due to the better implementation of the partial

simulation. The PFV time of the SC is 27 ms, that of MS is 12 ms, resulting in a performance gain of about double. This allows us to verify *Execute* unit in about half the time as compared to [16].

D. Limitations of the Approach

Although the proposed approach works well for a processor design like the one presented in this work, it has some limitations. BDD-based verification approach is not suitable for processors containing multiplier and divider units. We use a bit-level verification method that does not scale well for multipliers [32] or floating-point units. Although word-level methods exist like *Symbolic Computer Algebra* (SCA) based verification that can tackle multiplier verification as well, we have not considered this in the present work. This is considered to be a complex task and is aimed to be considered as a future work.

Also, while verifying a processor, certain design specifications must be provided by the designer, since a closed-box model would not give the verification engineer enough information about the processor to verify. Certain information is required to extract functionality of the entire processor properly. The current tools presented in this work uses BDD-based verification approach. The SSIM tool is limited to handling simple sequential circuits only. Improvement in the tools are required to accommodate more complex sequential circuits. The chosen intermediate format, viz. AIG, which bridges the gap between the Verilog representation of the processor and the BDD representation for verification also comes with certain limitations. Improved data structures are required to overcome such limitations.

V. CONCLUSION

In this paper, we present a BDD-based PFV technique to verify the MicroRV32 processor. Both combinational and sequential sub-circuits are included in the verification process. Firstly, the functionalities from the processor design are extracted and then the various units of the processor are verified. Different tools are developed to perform the complete verification process. A thorough experimentation has been performed to verify the processor design. It can be shown that the processor can be verified within polynomial bounds. This work includes the base instruction set of the processor for verification, and as a future work, M-extension could be incorporated where we target verification of more complex instructions like multiplication and division.

REFERENCES

- [1] J. Davis, A. Slobodová, and S. Swords, "Microcode verification - another piece of the microprocessor verification puzzle," in *Proc. Conf. Interact. Theorem Proving*, 2014, pp. 1–16.
- [2] S. Goel, A. Slobodová, R. Summers, and S. Swords, "Verifying X86 instruction implementations," in *Proc. Int. Conf. Certified Programs Proofs*, 2020, pp. 47–60.
- [3] M. Novak, I. Grobelna, U. M. Nyman, P. Szczesniak, and F. Blaabjerg, "Modular modeling and statistical validation for grid-connected FS-MPC-controlled matrix converters," *IEEE Trans. Ind. Electron.*, vol. 70, no. 9, pp. 8613–8623, Sep. 2023.
- [4] B. Steffen, *Methods, Languages and Tools for Future System Development*. Cham, Switzerland: Springer, 2019.
- [5] R. Drechsler, "PolyAdd: Polynomial formal verification of adder circuits," in *Proc. IEEE Symp. Des. Diagnostics Electron. Circuits Syst.*, 2021, pp. 99–104.
- [6] R. Drechsler and A. Mahzoon, "Polynomial formal verification: Ensuring correctness under resource constraints," in *Proc. Int. Conf. Comput.-Aided Des.*, 2022, Art. no. 70.
- [7] M. Schnieber, S. Fröhlich, and R. Drechsler, "Polynomial formal verification of approximate adders," in *Proc. EUROMICRO Symp. Digit. Syst. Des.*, 2022, pp. 761–768.
- [8] M. Schnieber, S. Fröhlich, and R. Drechsler, "Polynomial formal verification of approximate functions," in *Proc. IEEE Annu. Symp. VLSI*, 2022, pp. 92–97.
- [9] J. Kleinekathöfer, A. Mahzoon, and R. Drechsler, "Polynomial formal verification of floating point adders," in *Proc. Conf. Des., Automat. Test Europe*, 2023, pp. 1–2.
- [10] M. Nadeem, J. Kleinekathöfer, and R. Drechsler, "Polynomial formal verification of adder circuits using answer set programming," in *Proc. Reed-Muller Workshop*, 2023, pp. 1–8.
- [11] A. Konrad, C. Scholl, A. Mahzoon, D. Große, and R. Drechsler, "Divider verification using symbolic computer algebra and delayed don't care optimization: Theory and practical implementation," in *Formal Methods in System Design*. Berlin, Germany: Springer, 2024, pp. 1–37.
- [12] J. R. Burch, "Using BDDs to verify multipliers," in *Proc. Des. Automat. Conf.*, 1991, pp. 408–412.
- [13] M. Barhoush, A. Mahzoon, and R. Drechsler, "Polynomial word-level verification of arithmetic circuits," in *Proc. ACM IEEE Int. Conf. Formal Methods Models Codesign*, 2021, pp. 1–9.
- [14] J. Kumar, Y. Miyasaka, A. Srivastava, and M. Fujita, "Formal verification of integer multiplier circuits using binary decision diagrams," *IEEE Trans. Comput. Aided Des. Circuits Syst.*, vol. 42, no. 4, pp. 1365–1378, Apr. 2023.
- [15] R. E. Bryant, "Graph-based algorithms for Boolean function manipulation," *IEEE Trans. Comput.*, vol. 35, no. 8, pp. 677–691, Aug. 1986.
- [16] L. Weingarten, A. Mahzoon, M. Goli, and R. Drechsler, "Polynomial formal verification of processor: A RISC-V case study," in *Proc. Int. Symp. Qual. Electron. Des.*, 2023, pp. 1–7.
- [17] L. Weingarten, K. Datta, and R. Drechsler, "PolyMir: Polynomial formal verification of the MicroRV32 processor," in *Proc. ACM Int. Symp. Nanoscale Archit.*, 2023, Art. no. 24.
- [18] L. Weingarten, K. Datta, A. Kole, and R. Drechsler, "Complete and efficient verification for a RISC-V processor using formal verification," in *Proc. Conf. Des., Automat. Test Europe*, 2024, pp. 1–6.
- [19] A. Waterman and K. Asanović, "The RISC-V instruction set manual; Volume I: Unprivileged ISA," in *SiFive Inc. and CS Division*. Berkeley, CA, USA: Univ. of California, Berkeley, 2019. [Online]. Available: <https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/riscv-spec.html>
- [20] A. Waterman and K. Asanović, "The RISC-V instruction set manual; Volume II: Privileged architecture," in *SiFive Inc. and CS Division*. Berkeley, CA, USA: Univ. of California, Berkeley, 2019. [Online]. Available: <https://five-embeddev.com/riscv-priv-isa-manual/Priv-v1.12/riscv-privileged.html>
- [21] S. Ahmadi-Pour, V. Herdt, and R. Drechsler, "The MicroRV32 framework: An accessible and configurable open source RISC-V cross-level platform for education and research," *J. Syst. Archit.*, vol. 133, 2022, Art. no. 102757.
- [22] C. Papon, "SpinalHDL: Scala based HDL," 2021. [Online]. Available: <https://github.com/SpinalHDL/SpinalHDL>
- [23] K. S. Brace, R. L. Rudell, and R. E. Bryant, "Efficient implementation of a BDD package," in *Proc. 27th ACM/IEEE Des. Automat. Conf.*, New York, NY, USA, 1991, pp. 40–45.
- [24] V. Patankar, A. Jain, and R. Bryant, "Formal verification of an ARM processor," in *Proc. VLSI Des.*, 1999, pp. 282–287.
- [25] P. Mishra and N. Dutt, "A methodology for validation of microprocessors using equivalence checking," in *Proc. IEEE Int. Workshop Microprocessor Test Verification*, 2003, pp. 83–88.
- [26] M. Keim, R. Drechsler, B. Becker, M. Martin, and P. Molitor, "Polynomial formal verification of multipliers," *Formal Methods Syst. Des.: Int. J.*, vol. 22, no. 1, pp. 39–58, 2003.
- [27] R. Drechsler, "PolyAdd: Polynomial formal verification of adder circuits," in *Proc. 24th Int. Symp. Des. Diagnostics Electron. Circuits & Syst.*, 2020, pp. 99–104.
- [28] R. Drechsler, A. Mahzoon, and L. Weingarten, "Polynomial formal verification of arithmetic circuits," in *Proc. Int. Conf. Comput. Intell. Data Eng.*, 2021, pp. 457–470.

- [29] “ABC: A system for sequential synthesis and verification,” 2018. [Online]. Available <https://people.eecs.berkeley.edu/alanmi/abc/>
- [30] C. Wolf, “Yosys open synthesis suite,” 2024. [Online]. Available: <https://yosyshq.net/yosys/>
- [31] F. Somenzi, “CUDD: CU decision diagram package release 2.7.0,” 2018. [Online]. Available: <https://github.com/ivmai/cudd>
- [32] J. R. Burch, “Using BDDs to verify multipliers,” in *Proc. 28th ACM/IEEE Des. Automat. Conf.*, New York, NY, USA, 1991, pp. 408–412.



Lennart Weingarten received the M.S. degree from the University of Bremen, Bremen, Germany. He is currently working toward the Ph.D. degree with the University of Bremen working in the DFG project “Reinhart Koselleck PolyVer”. He is currently with the Polynomial Formal Verification of complex circuits with special focus on RISC-V processors.



Kamalika Datta received the M.S. degree from the Indian Institute of Technology Kharagpur, India, in 2010, and the Ph.D. from the Indian Institute of Engineering Science and Technology, Shibpur, India, in 2014. She was an Assistant Professor with the National Institute of Technology Meghalaya, Shillong, India, from August 2014 to 2018, Postdoctoral Research Fellow with the Nanyang Technological University, Singapore from 2019 to 2020, and an Associate Professor with JIS University Kolkata from 2020 to 2021. She is currently a Research Assistant

with the University of Bremen, Germany from October 2021. She has authored or coauthored more than 100 papers in peer reviewed journals and conferences. Her research interests include logic design using emerging technologies, synthesis and optimization of reversible and quantum circuit, and embedded systems.



Rolf Drechsler (Fellow, IEEE) received the Diploma and Dr. phil. nat. degree in computer science from the Johann Wolfgang Goethe University Frankfurt am Main, Germany, in 1992 and 1995, respectively. He worked with the Institute of Computer Science, Albert-Ludwigs University, Freiburg im Breisgau, Germany, from 1995 to 2000, and with the Corporate Technology Department, Siemens AG, Munich, Germany, from 2000 to 2001. Since 2001, he has been a Full Professor and Head of the Group of Computer Architecture, Institute of Computer Science, University of Bremen, Bremen, Germany. In 2011, he also became the Director of the Cyber-Physical Systems Group with the German Research Center for Artificial Intelligence (DFKI), Bremen. Since 2018, he has been the Dean of the Faculty of Mathematics and Computer Science. His current research interests include the development and design of data structures and algorithms with a focus on circuit and system design. He is an ACM Fellow From 2008 to 2013, he was the Vice Rector for *Research and Young Academics* with the University of Bremen.